Cloud Computing for Machine Learning

Dockers and Containerization

Asad Norouzi

School of Software Design and Data Science

Agenda

- Containerization
- Docker
- Docker Architecture
- How Docker Works?
- Benefits of Using Docker
- Docker vs. Virtual Machines
- Common Docker Commands
- Summary
- Q&A

Containerization

Containerization is a lightweight form of virtualization that allows you to package an application along with its dependencies into a single unit called a container.

Ensures consistency across development, testing, and production environments.

Solves the "it works on my machine" problem.

Containers share the host system's OS kernel but are isolated from each other and the host, making them lightweight compared to virtual machines.

Docker

Docker is an open-source platform that automates the deployment, scaling, and management of containerized applications.

Simplifies the creation and management of containers.

Provides tools to build, ship, and run applications consistently across environments.

Think of containers as shipping containers: no matter what’s inside, they can be easily transported and handled by standardized tools.

Docker Architecture

Docker Engine: Core component for building and running containers.

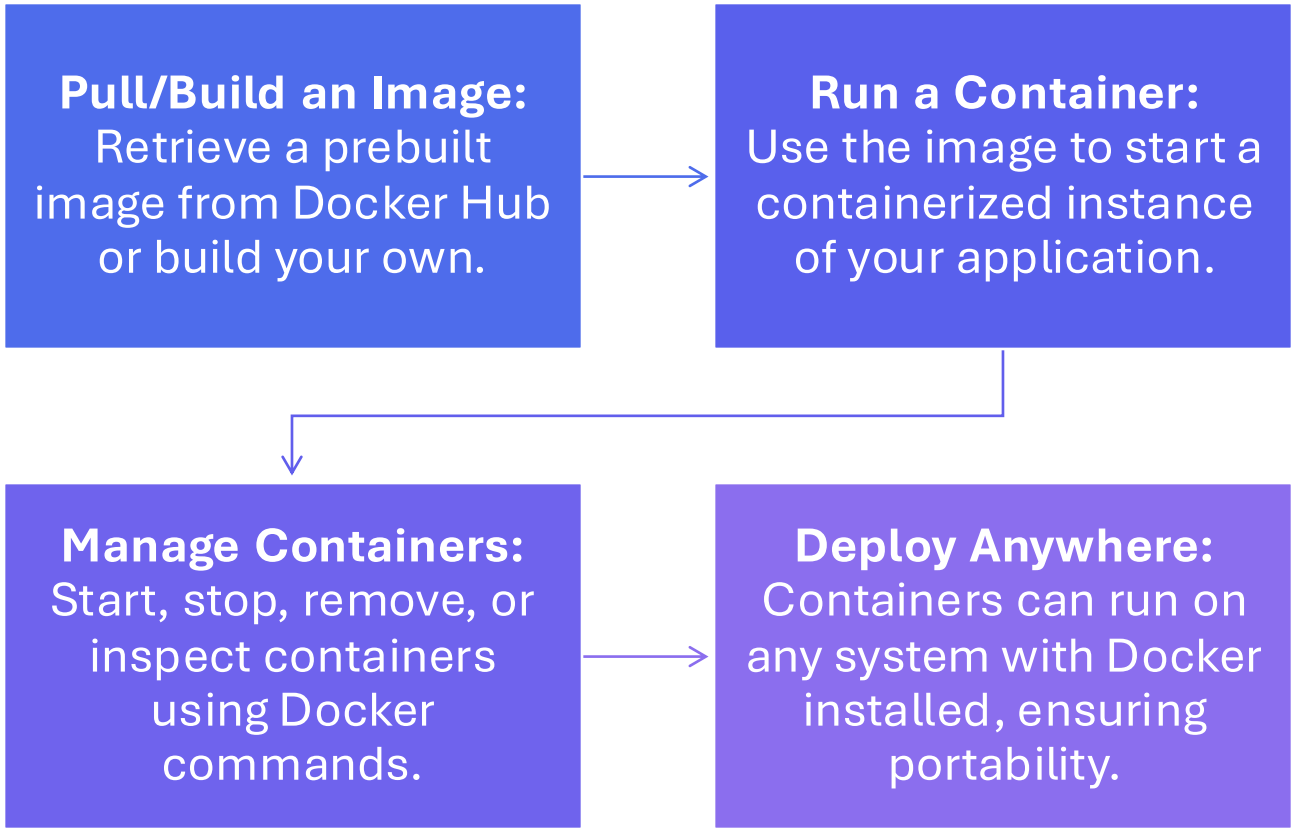
Images: Templates for creating containers (like snapshots of applications).

Containers: Running instances of Docker images.

Docker Hub: A cloud-based registry to share Docker images.

Volumes: Persistent storage for data used by containers.

How Docker Works?



Benefits of Using Docker

Portability:

Write once, run anywhere. Docker containers are platform-agnostic.

Efficiency:

Containers use fewer resources than virtual machines.

Scalability:

Easily replicate containers to scale applications.

Speed:

Quick to start, stop, and deploy compared to traditional environments.

Consistency:

Ensures the same environment from development to production.

Docker vs. Virtual Machines

Feature	Docker (Containers)	Virtual Machines
OS Sharing	Shared host OS kernel	Dedicated OS per instance
Resource Efficiency	Lightweight	Resource-heavy
Startup Time	Seconds	Minutes
Isolation Level	Process-level	Full hardware emulation

Common Docker Commands

Download	docker pull <image>: Download an image from Docker Hub.
Create and start	docker run <image>: Create and start a container from an image.
List	docker ps: List running containers.
Stop	docker stop <container_id>: Stop a running container.
Remove	docker rm <container_id>: Remove a stopped container.
Build	docker build -t <image_name> . : Build an image from a Dockerfile.

Examples

```
# Build the image
docker build -t my-python-app .

# Run the container
docker run -p 5000:5000 my-python-app
```

```
# Use official Python image
FROM python:3.8-slim

# Set working directory
WORKDIR /app

# Copy application files
COPY . /app

# Install dependencies
RUN pip install -r requirements.txt

# Run the application
CMD ["python", "app.py"]
```

Summary

Docker and containerization streamline software development and deployment.

Containers are lightweight, portable, and efficient compared to traditional virtual machines.

Docker provides a robust ecosystem for building, sharing, and running containers.

Mastering Docker commands and Dockerfiles is crucial for leveraging its full potential.

Q&A